

CAIGE · UNIVALE

Regras de Desenvolvimento

Normas obrigatórias para manutenção do frontend, componentes globais, responsividade, API, PWA e sincronização documental do CAIGE.

Regra máxima: se duas telas precisam da mesma coisa, essa coisa não pertence à tela.

Fonte da verdade

Design System + arquivos globais

Sem duplicação

Reutilizar grid, paginação, menus e Shell

Segurança e contrato

Backend e OpenAPI revisados no mesmo ciclo

V1 consolidada

PWA, uploads e documentação sincronizados



Sumário

Regras oficiais de implementação e manutenção do CAIGE.

1. Antes de alterar código

2. Fonte da verdade

3. Proibições

4. Tokens

5. Grid

6. Paginação

7. Redimensionamento de colunas

8. Responsividade

9. JavaScript

10. HTML

11. Nomenclatura

12. Alterações globais

13. CSS específico de página

14. !important

15. Compatibilidade com legado

16. Critério de conclusão

17. Regra máxima

18. Shell Global

19. PWA e dados sensíveis

20. Sincronização documental



Princípios obrigatórios

Resumo operacional antes de qualquer alteração.

1. Ler antes de alterar

Design System, Regras do Projeto e arquivos reais envolvidos devem ser inspecionados antes da implementação.

2. Reutilizar antes de criar

Grid, paginação, menu de ações, Shell, tokens e comportamentos compartilhados já possuem fontes canônicas.

3. Evitar atalhos locais

Não esconder problemas globais com CSS específico, inline style ou novos !important.

4. Validar desktop e mobile

Alterações compartilhadas devem ser testadas nos fluxos e perfis impactados.

5. Backend é a autoridade

Permissões e escopo não podem depender apenas da interface.

6. Documentar no mesmo ciclo

Contrato, PWA, schema, regras e padrões visuais devem permanecer sincronizados.



Estas regras são obrigatórias para qualquer pessoa ou IA que altere o frontend.

1. Antes de alterar código

1. Ler `DESIGN_SYSTEM.md`.
 2. Ler este arquivo.
 3. Inspecionar os arquivos reais envolvidos.
 4. Procurar componente global existente antes de criar outro.
 5. Informar quais arquivos serão alterados.
 6. Informar se a mudança é global ou específica.
 7. Não implementar até existir autorização explícita quando o fluxo de trabalho exigir aprovação.
-



2. Fonte da verdade

A ordem de responsabilidade é:

```
base.css  
→ layout.css  
→ componentes.css  
→ responsive.css  
→ paginas/*.css
```

CSS de página não deve substituir silenciosamente um componente global.



3. Proibições

Não criar, sem justificativa técnica:

- segunda implementação de grid;
 - segunda paginação;
 - segunda barra de ações;
 - segunda implementação de menu `:`;
 - zebra local;
 - cabeçalho de grid local;
 - fonte local de grid;
 - hover local de grid;
 - cor duplicada quando já existe token;
 - Google Fonts carregado diretamente em HTML (a fonte oficial é centralizada em `base.css`);
 - bloco `<style>` dentro de HTML;
 - `style=""` para layout permanente;
 - `!important` como solução de conflito;
 - regras globais dentro de CSS de página;
 - arquivos globais novos para algo já coberto pela arquitetura atual.
-



4. Tokens

Valores compartilhados de:

- cor;
- fonte;
- peso;
- radius;
- sombra;
- espaçamento;
- grid;

devem usar tokens já existentes ou, se realmente necessário, receber novo token em `base.css`.

Novo token deve ter finalidade compartilhada, não servir apenas para esconder um valor local.



5. Grid

Toda nova listagem tabular deve primeiro avaliar:

```
.data-table  
ou  
.data-grid
```

O padrão visual deve vir de `componentes.css`.

No desktop, toda listagem de registros/dados repetitivos deve usar o padrão global `data-table` ou `data-grid`. No mobile, a mesma estrutura pode ser reorganizada em cards responsivos por CSS, sem criar uma implementação paralela de lista. Usuários, Cursos, Atividades e demais listagens administrativas seguem esta regra.

CSS específico pode declarar a proporção inicial das colunas, mas não duplicar a aparência do componente.

No desktop, o padrão atual é centralizado.

Antes de criar uma nova listagem, classifique o conteúdo como:

- TABULAR SEMÂNTICO;
- OPERACIONAL TABULAR;
- LISTA DE ENTIDADES.

Se o conteúdo for percebido como bloco/entidade e não como linha/coluna, não forçar `data-table` nem `data-grid`.



6. Paginação

Listagens extensíveis devem reutilizar:

```
Frontend/recursos/js/paginacao.js
```

e a classe:

```
PaginacaoLista
```

Opções padrão:

```
10, 20, 30, 50, 100
```

Não criar paginação inline em cada tela.

A paginação não define a estrutura visual da lista. Ela pode acompanhar `data-table`, `data-grid` ou lista de entidades.



7. Redimensionamento de colunas

Usar:

```
Frontend/recursos/js/redimensionar-grid.js
```

Ativação:

```
data-grid-redimensionavel="true"  
data-grid-id="..."
```

Não criar listener de resize de coluna específico para tela.



8. Responsividade

Regra compartilhada:

```
responsive.css
```

Regra exclusiva da página:

```
paginas/<pagina>.css
```

Não mover regra exclusiva para `responsive.css` apenas para “funcionar”.



9. JavaScript

Antes de criar função nova para comportamento comum, procurar implementação global.

Preferir uma API compartilhada em vez de copiar lógica entre páginas.

Exemplos atuais:

```
PaginacaoLista  
RedimensionadorGrid  
menu-acoes.js  
notificacoes.js  
util-jwt.js  
EstadoSessao (estado-sessao.js)
```

Estado temporário de trabalho que precise sobreviver à navegação na mesma guia deve utilizar `EstadoSessao` ; não realizar acesso direto duplicado a `sessionStorage` nas páginas.



10. HTML

Novas páginas internas devem carregar, quando aplicável:

```
base.css
componentes.css
layout.css
responsive.css
paginas/<pagina>.css
```

Não carregar arquivos CSS legados removidos.

Não carregar fontes individualmente no HTML. A família oficial da interface é `IBM Plex Sans`, carregada exclusivamente por `base.css`, com fallback `Segoe UI`, `Arial`, `Helvetica`, `sans-serif`.

Não declarar `font-family` própria em CSS de página. Tamanhos compartilhados devem consumir os tokens tipográficos de `base.css` / `responsive.css`; não criar escalas locais para desktop, tablet ou mobile.

Não usar `<style>` embutido para construir a interface.



11. Nomenclatura

Código novo deve usar PT-BR quando tecnicamente aplicável:

- entidades;
- tabelas/colunas próprias do projeto;
- rotas próprias;
- variáveis;
- funções;
- nomes de arquivos novos;
- comentários;
- textos de interface.

Identificadores de bibliotecas externas permanecem como definidos pela biblioteca.



12. Alterações globais

Antes de alterar `base.css`, `layout.css`, `componentes.css`, `responsive.css` ou JS global:

1. identificar todas as páginas que usam o componente;
 2. evitar corrigir somente uma tela por seletor mais específico;
 3. testar Dashboard, Movimentações e Pacientes como referências;
 4. testar mobile quando a alteração puder afetar responsividade.
-



13. CSS específico de página

Um arquivo `paginas/*.css` só pode conter:

- estrutura exclusiva daquela tela;
- proporção específica de colunas;
- componente realmente exclusivo;
- composição visual exclusiva, como Login;
- regras responsivas exclusivas da tela.

Se uma regra aparecer em duas páginas, ela é candidata a componente global.



14. !important

Não adicionar novo !important sem:

- explicar por que a cascata normal não resolve;
- verificar se existe duplicação/legado causando o conflito;
- documentar a exceção.

A meta é reduzir os !important existentes durante as migrações.



15. Compatibilidade com legado

As telas revisadas da V1 usam os padrões globais consolidados. Não tratar código antigo ou uma implementação isolada como referência apenas por ainda existir no histórico do projeto.

Se uma tela ou trecho legado for identificado futuramente:

- não copiar o CSS legado para telas novas;
 - não considerar o legado como referência visual;
 - migrar o comportamento para os componentes e arquivos globais correspondentes;
 - remover referências a CSS inexistentes/antigos;
 - remover CSS inline desnecessário;
 - não reintroduzir carregamento individual de fontes.
-



16. Critério de conclusão

Uma alteração visual só está concluída quando:

- funciona;
 - mantém o padrão visual;
 - não cria implementação paralela;
 - passa pela auditoria;
 - não quebra desktop;
 - não quebra mobile quando aplicável.
-



17. Regra máxima

Se duas telas precisam da mesma coisa, essa coisa não pertence à tela.



18. Shell Global

O Shell Global é a arquitetura consolidada das páginas autenticadas.

Separação oficial:

GLOBAL

→ estrutura da aplicação

PÁGINA

→ conteúdo específico

Regras obrigatórias:

- páginas autenticadas devem consumir `Frontend/recursos/js/shell-global.js` ;
- não criar segunda sidebar, segundo cabeçalho, segundo menu do usuário ou quick-nav local em página autenticada;
- rotas do menu devem ter uma fonte canônica no Shell Global;
- páginas filhas devem usar `rotasRelacionadas` quando necessário;
- o item ativo da sidebar não deve ser hardcoded no HTML;
- o menu do avatar é apenas para ações pessoais;
- `Frontend/paginas/autenticacao/entrar.html` não usa o Shell autenticado.

Qualquer alteração estrutural que impacte navegação, header, sidebar, avatar ou quick-nav deve evoluir o Shell Global ou o módulo responsável, e não ser escondida em CSS/HTML local.

- O mesmo recurso funcional exibido para perfis diferentes deve reutilizar o mesmo componente visual global; diferenças entre Supervisor e Professor ficam restritas a permissões, escopo e ações autorizadas, não a uma segunda apresentação visual.



19. PWA e dados sensíveis

- `Frontend/service-worker.js` deve continuar limitado a recursos estáticos da interface.
 - Nunca adicionar `/api/` ou `/recursos/uploads/` ao cache do Service Worker.
 - Alterações em `manifest.webmanifest`, Service Worker, `pwa.js` ou ícones devem ser testadas em contexto instalável e sem quebrar o acesso web comum.
 - Fotos de pacientes usam o fluxo existente de `paciente-foto-editor.js` e `/api/pacientes/{id}/foto`; não criar armazenamento paralelo em `localStorage`, Cache Storage ou Base64 persistente no frontend.
-



20. Sincronização documental

- Mudança de comportamento, contrato HTTP, permissão, schema, PWA ou padrão visual documentado exige atualização da documentação correspondente no mesmo ciclo.
- `docs/api/openapi.yaml` continua sendo a fonte do contrato HTTP observável.
- `DESIGN_SYSTEM.md` continua sendo a fonte normativa visual; PDFs são versões de consulta e devem ser regenerados quando a fonte Markdown mudar de forma relevante.
- Manuais ilustrados devem ser revistos quando uma alteração de interface tornar prints ou instruções obsoletos.